Methodology and Computing in Applied Probability (2023) 25:3
https://doi.org/10.1007/s11009-023-09996-z

Optimal Liquidation Through a Limit Order Book: A Neural Network and Simulation Approach

Alexandre Roch[1]

Received: 30 March 2022 / Revised: 24 October 2022 / Accepted: 28 November 2022 /
Published online: 27 January 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
We present a learning algorithm based on simulation and neural networks to solve a stochastic optimal control problem with a large state space using dynamic programming. The problem consists in liquidating a given number of shares of a stock through a limit order book (LOB). The state space includes prices and quantities at each level in the LOB. The objective is to maximize the expected liquidation proceeds. Shares are sold by market orders matching the current limit orders in the LOB and have an impact on the future evolution of the LOB. The optimal strategy is obtained using a hybrid form of performance and value iteration procedures based on neural networks. The probability distribution of the LOB is estimated by a deep learning classification task. The model is tested on 12 stocks traded through the NYSE Arcabook, and a numerical implementation shows that the model outperforms the most common optimal liquidation models in the literature by a significant amount.

Keywords Stochastic control problem · Optimal liquidation · Limit order books · Neural networks

MSC Classification 91G60 · 93E20

1 Introduction

Optimal liquidation problems consist in finding the best trading strategy to sell a fixed number of shares of a financial asset while taking into account the costs incurred by trading in markets in which supply and demand is limited, trading counterparties are not always available, and trading causes price impacts that may or may not be permanent. Liquidity becomes a risk factor when these costs and their causes vary randomly through time. When trading is done through a limit order book (LOB), the current state of the order book offers a lot of information about these trading costs and its stochastic evolution may be modelled as a proxy for liquidity risk. However, the complexity of the information and its many

✉ Alexandre Roch
roch.alexandre_f@uqam.ca

[1] Faculty of Management, University of Quebec at Montreal, Montreal, Canada

dimensions makes the modelling exercise difficult and the search for an optimal trading strategy that takes the information into account even more difficult. In this paper, we present a numerical algorithm based on neural networks and dynamic programming that aims to represent this information and uses it to determine optimal liquidation strategies.

Optimal liquidation problems date back to Almgren and Chriss (1999) and the literature has been growing ever since. In their seminal paper, Almgren and Chriss propose a simple price impact setup and solve the optimal solution with a mean-variance objective. They consider both permanent and temporary price impacts and obtain a simple deterministic solution for the optimal speed of trading. Obizhaeva and Wang (2013) propose a liquidity model with a price impact that decays with time, a form of resilience of the order book. They also obtain a simple deterministic liquidation strategy in this framework. Schied and Schöneborn (2009) extend the Almgren-Chriss setup and find the optimal solution for a von Neumann-Morgenstern investor over an infinite horizon. Lokka and Xu (2020) revisit the Almgren-Chriss framework where the unaffected asset price is a Lévy process.

The models previously mentioned propose fixed parameters for the cost and the impact of a trade. However, other authors have proposed different modelling designs to consider the case of liquidity changing through time. Almgren (2012) extends the Almgren-Chriss model with a stochastic price impact parameter. Becherer et al. (2018) characterize the solution of an optimal liquidation problem in which the resilience of the price process and the impact of trades has an added noise component. The model by Alfonsi et al. (2010) is one of the few ones that consider a general shape for the order book. However, the shape is fixed and the only source of randomness is the midprice. Ackermann et al. (2021) consider an LOB model in which both order book depth and resilience are allowed to evolve randomly in time. Sadoghi and Vecer (2022) propose a piecewise deterministic Markov decision process (MDP) solution to the liquidation problem in which the price impact is a linear function of a self-exciting dynamic process representing incoming orders.

The papers discussed so far involved cases in which the investor is a price taker. However, many authors have also considered the liquidation problem with the possibility of posting limit orders and waiting for them to be filled. Cartea and Jaimungal (2015) determine the optimal liquidation schedule in which the liquidity cost of a market order is fixed but the probability that another market order matches the posted limit order is a function of the spread between the limit price and the midprice, modeled as a Brownian motion plus drift. Cartea et al. (2014) develop a complex high-frequency trading strategy in an order book characterized by feedback effects and adverse selection risk due to the presence of informed traders. Guilbaud and Pham (2013) determine the optimal trading strategy of a market maker who makes both types of trades and seeks to maximize the expected utility over a short term horizon. Chevalier et al. (2016) consider spread dynamics which are impacted by both limit and market orders of the implemented strategy. They present a numerical experiment calibrated on real LOB data. Further extensions and modelling variations in the optimal liquidation literature are put forward by Bayraktar and Ludkovski Bayraktar and Ludkovski (2014), Guéant and Lehalle (2015), and Guéant et al. (2011), to name a few. Many other optimal liquidation and market-making problems are presented in the book by Cartea et al. (2015).

The main modelling paradigm in the aforementioned papers is the use of a proxy to describe liquidity and/or liquidity risk, represented by constants or more generally by a one-dimensional stochastic process. Yet, the reality of trading is of course a lot more complex. This is even more apparent when one considers trading in a market driven by limit orders. In order to get a more realistic dynamic model of this kind of financial market, one must necessarily use a large number of variables and stochastic processes. For example, Cont et al.

(2010) propose a continuous-time stochastic model for order books and show that the information contained in the order book affects the probabilities of various events like trade execution and price movements. Sirignano (2019) uses deep neural networks to model spatial distributions of the information and price movements deep in the LOB. Nevmyvaka et al. (2006) used reinforcement learning and MDP to solve the optimal liquidation problem in which LOB information is summarized in low-resolution features. Similarly, Akbarzadeh et al. (2018) use an online model-based MDP approach with a one-dimensional state variable defined in terms of bid prices. In this paper, we tackle the problem of optimal liquidation in an LOB by allowing a more comprehensive modelling approach of the order book and by solving the problem using Monte Carlo simulation techniques and neural networks. The main advantage of using neural networks is to let the algorithm identify the relevant features of liquidity in the LOB and determine the resulting optimal strategy rather than leaving this task to the modeler. By testing the model with real-life data, we show that this more refined use of the information contained in the order book leads to substantial gains in comparison to other aforementioned reduced-form models of the order book in the context of optimal liquidation.

The optimal strategy is obtained using a hybrid form of performance and value iteration procedures. For the simulation part of the algorithm, the probability distribution of changes in the LOB is estimated from historical data by a deep learning classification task. The assumption is that a trading strategy has a direct impact on the LOB and its future evolution. The model must therefore be calibrated on simulated data. Furthermore, out-of-sample tests cannot be directly performed on historical data since every action of the optimal trading strategy would make the LOB diverge from its historical values. To circumvent this problem, we test the model on simulated data for which the probability distribution is calibrated from out-of-sample historical data using the same classification learning task than for in-sample data.

The use of neural networks in conjunction with dynamic programming was suggested early on by Bertsekas and Tsitsiklis (1995) in a survey of the subject. Reinforcement learning methods based on dynamic programming like Q-learning (Tsitsiklis (1994), Watkins and Dayan (1992) and temporal difference (TD) learning (Sutton (1988), Tesauro (1992) can often times be improved by the use of neural networks. More recently, reinforcement learning has successfully been applied to option pricing and hedging (Carbonneau and Godin (2021), Marzban et al. (2021)), algorithmic trading (Cartea et al. (2021)) and portfolio optimization (Tamar et al. (2015)). However, as it is often the case in financial applications, the time variable in optimal liquidation problems plays a crucial role in the optimal solution. In this sense, the algorithm we present in this paper deviates in its use of dynamic programming from typical Q-learning and TD methods in that action-state value functions are approximated by backward induction in time. In other words, time is not considered a state variable. The method is more akin to approximate dynamic programming approaches typically found in the financial engineering literature.

Approximate dynamic programming approaches typically fall in two categories: value iteration and performance iteration methods. In the former, a simulation and regression procedure is used to estimate the value function at each time step. In the latter, the optimal policy is approximated and the value function is estimated using trajectories that are simulated using the approximate optimal policy computed at each step of the algorithm. The problem of pricing American options has been extensively studied using these regression-based Monte Carlo simulation approaches (Belomestny et al. (2018), Broadie et al. (2000), Egloff et al. (2007), Glasserman and Yu (2004), Longstaff and Schwartz (2001), Stentoft (2004), Tsitsiklis and Van Roy (2001)). In this setting, the only control is a stopping time and it does not affect the

state process. Carmona and Ludkovski (2008) study an impulse control problem in the jump-diffusion case by proposing a method based on a sequence of optimal impulse boundaries. The problem becomes more difficult when the state process is affected by the control. Regression-based methods have also been used for these more general stochastic control problems. Kharroubi et al. (2014) propose a numerical algorithm based on control randomization in the diffusion case. In the spirit of value iteration, Denault et al. Denault et al. (2017), Denault and Simonato (2017) include the control as an independent variable in the regression part of the algorithm in order to take into account the impact of the control on the state process. Denault et al. (2013) present another value-iteration algorithm in the case of a control process that can only take three values and when the only endogenous (controlled) variable is deterministic and is only affected by the control. Gyurko et al. (2015) use a dual formulation of the problem to construct a value-iteration algorithm using the action as independent variable in the regression. Huang and Kwok (2016) present another regression-based Monte Carlo method of the value-iteration type for variable annuities with guarantees. Their method relies on the fact that the optimal control is of the bang-bang type which reduces the optimal policies to the three choices. Barrera-Esteve et al. (2006) and Han and Ee (2016) present algorithms based on the direct representation of the optimal control by neural networks.

Recent advances have demonstrated the practicality of neural networks and simulation-based methods in other financially motivated problems. Bachouch et al. (2022) and Huré et al. (2021) present theoretical convergence results of several deep-learning algorithms with applications to option hedging and energy storage problems. Buehler et al. (2019) present a deep reinforcement learning algorithm for hedging a large portfolio of derivatives. We refer the reader to Germain (2021) for an up-to-date review of machine learning techniques for financial applications. Our hybrid approach consists in estimating the conditional expectation of the value function one step ahead by a performance iteration procedure in which the optimal policy is deduced directly from the estimated value function, instead of representing it through a parameterized family of functions and without including the control in the regression. The approach relies on the assumption that, at each time step, the random controlled evolution of the state process occurs in two steps. First, the control has an impact on the current state. Then, the probability distribution of the state at the next time step only depends on the post-decision state variable. This approach is particularly useful in a setting in which the control space is finite, low-dimensional and discrete.

The rest of the paper is organized as follows. The optimization problem and the main variables representing the LOB are defined in the following section. The dynamics of the LOB are presented in Section 3. The dynamic programming principle is formulated in Section 5 and the neural network algorithms based on this principle are presented in Section 6. The paper concludes with a numerical implementation of the model on 12 different firms traded on the NYSE Arcabook. The calibration procedure is explained in Section 7 whereas numerical results and benchmark results are found in Section 8. Average liquidation costs, risk-adjusted costs and value at risk are computed. Section 9 concludes the paper.

2 The Modelling of the Order Book and the Stochastic Optimization Problem

A limit order book presents all limit orders submitted by market participants indicating prices that participants are willing to pay or ask for the asset, and their associated quantities. The best bid price represents the highest price that market participants are willing to

pay in exchange for the asset at a given time. Other bid prices are then presented in levels below the best bid price. All limit orders at a given price are aggregated and the total quantity is presented in the associated level of the LOB.

We take the point of view of an investor who owns N_0 shares of the asset and aims to liquidate[1] them through market orders. The investor observes the LOB and chooses a liquidation strategy in order to maximize the expected liquidation proceeds.

We define B_t as the best bid price. Limit orders are aggregated in terms of prices and we denote by π_t^d the available aggregate quantity in the d-th level of the LOB, at time t. We assume a maximum of $m+1$ levels including the current best bid price. The price associated to the d-th level of the LOB is then $B_t - d\varsigma$, in which ς is the minimum tick (typically 1 cent). Level 0 corresponds to the best bid price. By definition, we automatically set $\pi_t^d = 0$ if $B_t - d\varsigma < 0$.

We denote by $Q_t^d = \sum_{j=0}^{d} \pi_t^d$, the cumulative available quantity up to level d. We also write $\overrightarrow{Q}_t$ for the (horizontal) vector $(Q_t^0, Q_t^1, \dots, Q_t^m)$. The order book can therefore be fully represented by the best bid price B_t and the vector $\overrightarrow{Q}_t$ at time t.

The investor must liquidate her portfolio before time T. She does so by submitting market orders that are matched against prevailing limit orders present in the order book. A single market order may go through more than one level. The market order is then matched with the quantity available at each best level starting with the best bid, until the order is completed. A market order of size $x \geq 0$ therefore depletes all the levels up to the level d that satisfies $Q_t^d \leq x < Q_t^{d+1}$. From this observation, we conclude that the proceeds of a sale of $x \geq 0$ shares at time t are denoted by $S(\overrightarrow{Q}_t, B_t, x_t)$ in which

$$\begin{aligned} S(\vec{q}, b, x) =& b\big(q^0 1_{x>q^0} + x 1_{x\leq q^0}\big) \\ &+ \sum_{d=1}^{m} (b - d\varsigma)\big((q^d - q^{d-1}) 1_{x>q^d} \\ &+ (x - q^{d-1}) 1_{q^{d-1}<x\leq q^d}\big), \end{aligned} \tag{1}$$

if $x \leq q^m$. For $x > q^m$, we define $S(\vec{q}, b, x) = S(\vec{q}, b, q^m)$, so that any shares in a number exceeding q^m is sold at a price of 0. We let x_t denote the number of shares sold by the investor at time t. The *total liquidation proceeds* are then given by

$$\sum_{t=1}^{T} S(\overrightarrow{Q}_t, B_t, x_t).$$

3 Dynamics of the LOB

The important missing modeling ingredient so far is the evolution of the bid prices and the impact of past trades on them. The pair of processes $(B, \overrightarrow{Q})$ is assumed to be a Markov decision process, affected by the actions x_t. The dynamics of the LOB from one period to another occurs in two steps. The first step corresponds to the impact of a trade on the levels of the LOB, which impacts the best bid price and the remaining levels of the LOB. In the

[1] The same setup can be used for an investor who aims to purchase N_0 shares through an LOB by considering ask prices instead of bid prices.

second step, the stochastic evolution of the levels are given by a Markovian transition probability distribution which only depends on the post-decision state variables (levels and bid price after the impact of a trade). The new best bid price is then identified and levels are relabeled appropriately. The values of the best bid price B_t and the levels $\overrightarrow{Q}_t$ are given at each step by $B_t^{(i)}$ and $\overrightarrow{Q}_t^{(i)}$ for $i = 1, 2$.

Recall that a market order of size x_t depletes parts of the LOB, so that immediately after the order at time t, the new best bid price is

$$B_t^{(1)} = B_t - L(\overrightarrow{Q}_t, x_t)\varsigma, \tag{2}$$

in which the function $L : \mathbb{N}^{m+1} \times \mathbb{N}_+$ is set to

$$L(\vec{q}, x) = \sum_{d=1}^{m} d\vec{1}_{q^{d-1} \leq x < q^d}.$$

Note that $B_t^{(1)} \geq 0$ when $x_t \leq Q_t^m$ because $\pi_t^j = 0$ if $B_t - j\varsigma < 0$.

Each level up to $L(\overrightarrow{Q}_t, x_t)$ is depleted after the market order x_t, so that the new best bid has an associated quantity $\pi_t^{0,(1)} = \pi_t^d - (Q_t^d - x_t)$ for $d = L(\overrightarrow{Q}_t, x_t)$, and each other level is thus relabeled:

$$\pi_t^{j,(1)} = \pi_t^{j+L(\overrightarrow{Q}_t, \tilde{x}_t)} \text{ for all } j \in \{1, \dots, m-d\}. \tag{3}$$

The remaining levels $m - d + 1 \leq j \leq m$, are then set to zero:

$$\pi_t^{j,(1)} = 0 \text{ for all } j \in \{m-d+1, \dots, m\}. \tag{4}$$

We then let

$$Q_t^{d,(1)} = \sum_{j=0}^{d} \pi_t^{d,(1)}, \tag{5}$$

representing the new aggregate levels of liquidity.

In the second step of the evolution of the LOB from time t to $t + 1$, new orders come in and leave the order book, having an impact on the quantity at each level. Furthermore, the best bid price can increase if a limit order is placed at a price higher than the current best bid price. Given the high frequency nature of orders in the LOB, we assume only one order enters at a given time. In particular, this means that only one level can change at a given time, which substantially reduces the number of outcomes. To describe this step of the evolution of the LOB, we introduce the random variables $\tilde{Q}_t$ and R_t representing the new quantity and level of the change at time t. R_t takes values in $\{-\overline{m}, -\overline{m}+1, \dots, 0, 1, \dots, m\}$. A negative value for R_t represents an incoming order at a level *above* the best bid price $B_t^{(1)}$. Here, $\overline{m} > 0$ represents the maximum number of levels above the current best bid at which there can be new incoming limit orders. The probability distribution of $(R_t, \tilde{Q}_t)$ is assumed to depend only on the post-decision state variables $\overrightarrow{Q}_t^{(1)}$ and $B_t^{(1)}$, and is independent of time. More specifically, there exist functions ϕ_R and ϕ_Q such that

$$\mathbb{P}(R_t = j \mid \overrightarrow{Q}_t^{(1)} = \vec{q}, B_t^{(1)} = b) = \phi_R(j; \vec{q}, b), \tag{6}$$

$$\mathbb{P}(\tilde{Q}_t = q \mid R_t = j, \overrightarrow{Q}_t^{(1)} = \vec{q}, B_t^{(1)} = b) = \phi_Q(q; j, \vec{q}, b), \tag{7}$$

$-\overline{m} \leq j \leq m$, $q \in \mathbb{N}$, $\vec{q} \in \mathbb{N}^{m+1}$, where $\mathbb{N}$ is the set of non-negative integers. At every time t, given $(\overrightarrow{Q}_t^{(1)}, B_t^{(1)})$, a random realization of R_t and $\tilde{Q}_t$ is drawn from $\phi_R(\cdot;\overrightarrow{Q}_t^{(1)}, B_t^{(1)})$ and $\phi_Q(\cdot;R_t, \overrightarrow{Q}_t^{(1)}, B_t^{(1)})$, respectively. We denote by $\overrightarrow{Q}_t^{(2)}$ the new quantities at each level (with respect to $B_t^{(1)}$), i.e.

$$Q_t^{j,(2)} = \begin{cases} Q_t^{j,(1)}, & \text{if } R_t \neq j, j \geq 0, \\ 0, & \text{if } R_t \neq j, j < 0, \\ \tilde{Q}_t, & \text{if } R_t = j. \end{cases} \tag{8}$$

for $-\overline{m} \leq j \leq m$. We then set $B_{t+1} = B_t^{(1)} - j^* \varsigma$ with j^* the lowest integer for which $\overrightarrow{Q}_t^{(2)}$ is non-zero. We then relabel levels accordingly:

$$\begin{aligned} Q_{t+1}^{0} &= Q_t^{j^*,(2)} \\ Q_{t+1}^{1} &= Q_t^{j^*+1,(2)} \\ &\cdots \\ Q_{t+1}^{m-j^*} &= Q_t^{m,(2)} \end{aligned}$$

The remaining Q_{t+1}^{j} (i.e. for $m - j^* < j \leq m$) are set to zero.

4 Estimation of the transition probabilities

Given the very large state space of all possible LOB configurations, we estimate the probability distribution of $\overrightarrow{Q}_t^{(2)}$ given $(\overrightarrow{Q}_t^{(1)}, B_t^{(1)})$ as in Eqs. (6) and (7) using a deep learning classification task. We first estimate the probability of a change in level j ($-\overline{m} \leq j \leq m$) and then the probability distribution of the new quantity, given a change in level j. These two quantities are estimated by training a classification neural network with 2 hidden layers of 15 nodes with cross-entropy loss function.

A steady state distribution of $\overrightarrow{Q}_t$ is estimated by simulating the LOB over a long period of time (1000 seconds) using $x_t = 0$.

4.1 Zero Drift Assumption

In the sample data, the average price movement over the sample and test periods can be negative or positive. The transition probabilities obtained from the above calibration procedure will inevitably produce similar price patterns in the Monte Carlo simulation. In order to prevent the algorithm to learn this drift component of prices and skew the optimal trading trajectory to benefit from it, we scale the probability of an upswing (i.e. the probability that a change occurs in levels above the current best bid) to make the average price movement equal to 0. More specifically, we compute the value ζ such that if ϕ_R is replaced by

$$\tilde{\phi}_R(j;\vec{q}, b) = \begin{cases} \zeta \phi_R(j;\vec{q}, b), & j < 0; \\ \frac{1-\sum_{i<0} \zeta \phi_R(i;\vec{q},b)}{\sum_{i\geq 0} \phi_R(i;\vec{q},b)} \phi_R(j;\vec{q}, b), & j \geq 0, \end{cases}$$

then

$$E\left(B_{t+1} - B_t\right) = 0,$$

in which the expectation is computed using the steady state distribution of $\overrightarrow{Q}_t$ and using the strategy $x_t = 0$. This allows us to obtain an optimal trading solution which is not biased by the training data.

5 The Optimal Control Problem and Dynamic Programming

A strategy $(x_s)_{s=t,\ldots,T}$ started at time t with N_t remaining shares is said to be admissible if there exists a (deterministic) policy function ϖ such that $x_t = \varpi(B_t, \overrightarrow{Q}_t)$, $x_t \geq 0$ for all t and $\sum_{s=t}^{T} x_s = N_t$. The set of admissible strategies is denoted $\mathcal{A}(n,t)$ when $N_t = n$. Given an admissible strategy $(x_s)_{s=t,\ldots,T}$, we define the remaining shares process by $N_s = N_t - \sum_{\tau=t}^{s-1} x_\tau$. The goal of the investor is to maximize the expected liquidation proceeds over admissible strategies. We therefore define the value function

$$v(n, \vec{q}, b, t) := \sup_{(x_s)\in\mathcal{A}(n,t)} \mathbb{E}\left(\sum_{s=t}^{T} S(\overrightarrow{Q}_s, B_s, x_s) \middle| \overrightarrow{Q}_t = \vec{q}, B_t = b \right),$$

in which S is given by Eq. (1).

We define the *total liquidation cost* of a strategy as the difference between the marked-to-market value of the initial position at time 0 and the total liquidation proceeds over $[0, T]$:

$$N_0 B_0 - \sum_{s=0}^{T} S(\overrightarrow{Q}_s, B_s, x_s).$$

The goal of the investor is therefore equivalent to minimizing the expected liquidation cost over admissible strategies.

We use dynamic programming (DP) to solve the above control problem:

$$v(n, \vec{q}, b, t) = \max_{x \leq q^n} \Big[S(\vec{q}, b, x)$$

$$+\mathbb{E}\left(v(n - x, \overrightarrow{Q}_{t+1}, B_{t+1}, t+1) \middle| \overrightarrow{Q}_t^{(1)}(x), B_t^{(1)}(x) \right) \Big], \tag{9}$$

for all $t < T$, in which $\overrightarrow{Q}_t^{(1)}(x)$ and $B_t^{(1)}(x)$ are controlled by action x according to Eqs. (2) and (5) starting with $\overrightarrow{Q}_t = \vec{q}$ and $B_t = b$. We also have the terminal condition

$$v(n, \vec{q}, b, T) = S(\vec{q}, b, n),$$

since at time T all remaining shares n must be liquidated. By defining the $\mathcal{Q}$-function (or state-action function):

$$\begin{aligned} \mathcal{Q}(n, \vec{q}, b, t; x) :=& S(\vec{q}, b, x) \\ &+ \mathbb{E}\left(v(n - x, \overrightarrow{Q}_{t+1}, B_{t+1}, t+1) \middle| \overrightarrow{Q}_t^{(1)}(x), B_t^{(1)}(x) \right), \end{aligned} \tag{10}$$

the dynamic programming principle can then also be written as follows:

$$v(n, \vec{q}, b, t) = \max_{x \le q^m} \mathcal{Q}(n, \vec{q}, b, t; x).$$

From a numerical perspective, the difficult part in the dynamic programming principle is to calculate the conditional expectation in Eq. (5). The expectation is conditioned on $m + 3$ state variables (excluding time), which take a very large number of values with nonlinear relations. To circumvent the curse of dimensionality, the proposed method consists in estimating each conditional expectation using neural network techniques by randomly sampling state variables by Monte Carlo methods and working backwards in time.

In low dimension problems, a typical dynamic programming procedure is to estimate the conditional expectation of v as given in Eq. (5) at each point in the domain, and then compute the value function v everywhere by calculating the maximum that appears in Eq. (5). This procedure therefore implies the estimation of two functions: the conditional expectation and the value function. To limit the compounding of errors that this estimation procedure entails, we make use of the $\mathcal{Q}$-function representation of Eq. (10) and only estimate the expectation at each time step. In this sense, we aim to estimate the function

$$\begin{aligned}
\mathcal{E}(n, \vec{q}, b, t) :=& \mathbb{E}\left(v(N_t, \overrightarrow{Q}_{t+1}, B_{t+1}, t+1) \middle| N_t = n, \overrightarrow{Q}_t^{(1)} = \vec{q}, B_t^{(1)} = b \right) \\
=& \mathbb{E}\left(\max_{x \le q^m} \mathcal{Q}(N_t, \overrightarrow{Q}_{t+1}, B_{t+1}, t+1; x) \middle| N_t = n, \overrightarrow{Q}_t^{(1)} = \vec{q}, B_t^{(1)} = b \right) \\
=& \mathbb{E}\Big(\max_{x \le q^m} \Big(S(\overrightarrow{Q}_{t+1}, B_{t+1}, x) \\
&\quad + \mathcal{E}(N_t - x, \overrightarrow{Q}_{t+1}^{(1)}(x), B_{t+1}^{(1)}(x), t+1) \Big) \Big| N_t = n, \overrightarrow{Q}_t^{(1)} = \vec{q}, B_t^{(1)} = b \Big),
\end{aligned} \tag{11}$$

for $t < T - 1$, using the dynamic programming principle for v and $\mathcal{Q}$. The terminal condition of $\mathcal{E}$ is

$$\mathcal{E}(n, \vec{q}, b, T-1) := \mathbb{E}\left(S(\overrightarrow{Q}_T, B_T, n) \middle| \overrightarrow{Q}_{T-1}^{(1)} = \vec{q}, B_{T-1}^{(1)} = b \right).$$

Note that we do not need to make $\mathcal{E}$ depend on the control x, unlike $\mathcal{Q}$. Starting at time t with $\overrightarrow{Q}_t = \vec{q}$, $B_t = b$ and n remaining shares, the expectation of the value function a time step later is $\mathcal{E}(n - x_t, \overrightarrow{Q}_t^{(1)}(x), B_t^{(1)}(x), t)$ when using control x_t at time t. From a numerical perspective, it allows us to eliminate the x-variable in the estimation procedure, in contrast to $\mathcal{Q}$-learning approaches that aim to approximation the state-action function, corresponding to $\mathcal{Q}$ in our setting. Given that we optimize over x and good accuracy of this dependence is needed, this aspect greatly improves the performance of the algorithm presented below.

6 The Simulation and Neural Network Algorithm

In this section, we present two algorithms based on simulation and approximation of the conditional expectation $\mathcal{E}$ using neural networks. The idea behind the approximation of $\mathcal{E}(n, \vec{q}, b, t)$ is based on the fact that the conditional expectation (as a random variable) is the minimizer of the least-squares minimization problem (least-squares projection). In other words, $\mathbb{E}(Y|Z) = g^*(Z)$ in which

$$g^* = argmin_g \mathbb{E}(Y - g(Z))^2,$$

where the minimum is taken over all measurable functions g. (See Glasserman and Yu (2004).) Following Eq. (11), we can take the paired data $Z = (N_t, \overrightarrow{Q}_t^{(1)}, B_t^{(1)})$ and

$$Y = \max_{x \le q^m} \left(S(\overrightarrow{Q}_{t+1}, B_{t+1}, x) + \mathcal{E}(N_t - x, \overrightarrow{Q}_{t+1}^{(1)}(x), B_{t+1}^{(1)}(x), t+1) \right)$$

to estimate $\mathcal{E}(n, \vec{q}, b, t)$, for fixed t. Given the discrete nature of the problem, we also have that $\mathbb{E}(Y|Z = z) = g^*(z)$ for all $z \in \mathbb{R}^{m+3}$. The algorithms presented below estimate the minimizers g^* by neural networks obtained by minimizing the mean squared error of the responses Y on the data Z.

6.1 Value Iteration

The first algorithm aims to estimate the expectation function $\mathcal{E}$ using a value iteration procedure. We start with time $t = T - 1$, and select a random sample of M $(m+3)$ -tuples: $z_i = (n_i, \vec{q}_i, b_i)$, $i = 1, \dots, M$. n_i and b_i are sampled from a uniform distribution, whereas $\vec{q}_i$ is sampled from the empirical steady-state distribution. For each $i \le M$, we let $(\overrightarrow{Q}_{t,i}^{(1)}, B_{t,i}^{(1)}) = (\vec{q}_i, b_i)$ evolve randomly according to the model presented in Section 3 with $x_t = 0$ to get $(\overrightarrow{Q}_{t+1,i}, B_{t+1,i})$. We then train a first neural network over the inputs z_i on the responses $S(n_i, \overrightarrow{Q}_{t+1,i}, B_{t+1,i}), i \le M$. We denote the network by $\mathrm{E}(n, \vec{q}, b; T-1)$.

We then proceed by backward induction with respect to the time variable t. For $t < T - 1$, we again select a random sample of M state variables $(m+3)$-tuples $z_i = (n_i, \vec{q}_i, b_i), i = 1, \dots, M$, and let it evolve randomly to get $Z_i = (n_i, \overrightarrow{Q}_{t+1,i}, B_{t+1,i})$, starting from $(\overrightarrow{Q}_{t,i}^{(1)}, B_{t,i}^{(1)}) = (\vec{q}_i, b_i)$, $i = 1, \dots, M$. To optimize over x, we augment Z_i to $\tilde{Z}_{i,j} = (n_i, \overrightarrow{Q}_{t+1,i}, B_{t+1,i}, x_j)$, $i \le M$, $x_j = 0, \dots, \bar{q}$, with $\bar{q}$ representing the maximum value of x allowed. For each sampled point $\tilde{Z}_{i,j}$ we compute the estimated Q-function and maximize over j:

$$\varrho_i = \max_j S(\overrightarrow{Q}_{t+1,i}, B_{t+1,i}, x_j) + \mathrm{E}(n_i - x_j, \overrightarrow{Q}_{t+1,i}^{(1)}(x_j), B_{t+1,i}^{(1)}(x_j); t+1), i \le M. \tag{12}$$

Finally, we train a neural network $\mathrm{E}(n, \vec{q}, b; t)$ over the inputs $(n_i, \vec{q}_i, b_i)$, on the responses ϱ_i, $i \le M$.

The first NNDP algorithm can be summarized as follows:

Algorithm 1 NNDP1

for $t = T-1$ to 1 **do**
  Sample $\vec{z}_i = (n_i, \vec{q}_i, b_i)$, $i = 1, \ldots, M$.
  Simulate $\vec{Z}_i = (n_i, \vec{Q}_{t+1,i}, B_{t+1,i})$, $i = 1, \ldots, M$, from transition distribution with $x_t = 0$, starting from $\vec{z}_i$.
  for $i \leq M$ **do**
    for $x_j = 0, \ldots, \bar{q}$ **do**
      Compute $\vec{Q}^{(1)}_{t+1,i}(x_j), B^{(1)}_{t+1,i}(x_j)$
    end for
    Compute

$$\varrho_i = \begin{cases} \max_j S(\vec{Q}_{t+1,i}, B_{t+1,i}, x_j) \\ \qquad +\mathrm{E}(n_i - x_j, \vec{Q}^{(1)}_{t+1,i}(x_j), B^{(1)}_{t+1,i}(x_j);\ t+1), & \text{if } t < T-1; \\ S(n_i, \vec{Q}_{t+1,i}, B_{t+1,i}), & \text{if } t = T-1. \end{cases}$$

  end for
  Train neural network $\mathrm{E}(\cdot;\ t)$ over inputs $(\vec{z}_i)$ with responses ϱ_i.
end for

6.2 Truncated Hybrid Performance Iteration Algorithm

Another common approach in the literature is the so-called performance iteration procedure in which trajectories are recomputed up to the terminal time using the optimal policy computed at each step of the algorithm. Instead of estimating $\mathcal{E}(\cdot, t)$ from $\mathcal{E}(\cdot, t+1)$, as in (11), in the performance iteration procedure we use the following representation of $\mathcal{E}$:

$$\mathcal{E}(n, \vec{q}, b, t) = \mathbb{E}\left(\sum_{s=t+1}^{T} S(\overrightarrow{Q}_s, B_s, X^*(N_s, \overrightarrow{Q}_s, B_s)) \middle| N_t = n, \overrightarrow{Q}^{(1)}_t = \vec{q}, B^{(1)}_t = b \right)$$

in which $X^*(N_s, \overrightarrow{Q}_s, B_s, s)$ is the optimal trade at time s given the current state of the order book $(\overrightarrow{Q}_s, B_s)$ and N_s shares remaining. However, instead of representing the optimal strategy by a neural network as it would naturally be done in a performance iteration procedure, the optimal strategy is deduced directly from the network E in this hybrid form of value and performance iteration procedures. It is given in terms of $\mathcal{E}$ by

$$X^*(N_s, \overrightarrow{Q}_s, B_s, s) = \begin{cases} \operatorname{argmin}_x S(\overrightarrow{Q}_s, B_s, x) + \mathcal{E}(N_s - x, \overrightarrow{Q}^{(1)}_s(x), B^{(1)}_s(x), s), & s < T; \\ N_s, & s = T. \end{cases}$$

Furthermore, since the simulation of the order book is very computationally and time intensive, we adapt the performance iteration procedure using the following dynamic programming principle for $\mathcal{E}$:

$$\begin{aligned}\mathcal{E}(n,\vec{q},b,t) =& \mathbb{E}\Big(\sum_{s=t+1}^{T'} S(\overrightarrow{Q}_s, B_s, X^*(N_s, \overrightarrow{Q}_s, B_s, s)) \\ &+ \mathcal{E}(N_{T'}, \overrightarrow{Q}_{T'}, B_{T'}, T')\Big| N_t = n, \overrightarrow{Q}_t^{(1)} = \vec{q}, B_t^{(1)} = b\Big),\end{aligned} \tag{13}$$

in which $T' = T \wedge (t+\omega)$ and $\omega \geq 1$ is the time window on which the policy iteration procedure is performed. Since the minimum is taken over a finite set, this procedure is in fact less computationally expensive than a typical performance iteration algorithm that aims to represent X^*.

As before, we can estimate the conditional expectations $\mathcal{E}(\cdot, t)$ of Eq. (13) with neural networks using the data $Z = (N_t, \overrightarrow{Q}_t^{(1)}, B_t^{(1)})$ and the responses

$$Y = \sum_{s=t+1}^{T'} S(\overrightarrow{Q}_s, B_s, X^*(N_s, \overrightarrow{Q}_s, B_s, s)).$$

The truncated hybrid performance iteration NNDP algorithm can be summarized as follows:

6.3 Implementation Parameters

We estimate $\mathrm{E}(\cdot, t)$ using deep neural networks with 2 hidden layers of 15 and 10 neurons, respectively. We use the Tan-sigmoid transfer function, and the output layer is linear. Training is done by minimizing the mean squared error with the Levenberg-Marquardt backpropagation method. Parameters of net $\mathrm{E}(\cdot, t+1)$ are reused as starting point for training net $\mathrm{E}(\cdot, t)$. The size of the sampling of state variables is 200 000 at each point in time, of which 40 000 is used for validation. The training of the networks stops after 5 failed validations to prevent the algorithm from overfitting to the training dataset. In the second algorithm, the time window ω is set to 10.

Algorithm 2 NNDP2

for $t = T - 1$ to 1 **do**
  Set $t' = T \wedge (t + \omega)$.
  Sample $\vec{z}_i = (n_i, \vec{q}_i, b_i)$, $i = 1, \ldots, M$.
  Simulate $\vec{Z}_i = (n_i, \vec{Q}_{t+1,i}, B_{t+1,i})$, $i = 1, \ldots, M$, from transition distribution with $x_t = 0$, starting from $\vec{z}_i$.
  for $i \leq M$ **do**
    for $s = t + 1$ to t' **do**
      Compute $(\vec{Q}^{(1)}_{s,i}(x_{s,i}), B^{(1)}_{s,i}(x_{s,i}))$ using

$$x_{s,i} = \begin{cases} \operatorname{argmax}_{x_j} S(\vec{Q}_{s+1,i}, B_{s+1,i}, x_j) & \\ \qquad + \mathrm{E}(N_{s,i} - x_j, \vec{Q}^{(1)}_{s+1,i}(x_j), B^{(1)}_{s+1,i}(x_j);\ s+1), & s < T; \\ N_{s,i}, & s = T. \end{cases}$$

    end for
    Compute

$$\varrho_i = \sum_{s=t+1,\ldots,t'} S(\vec{Q}_{s,i}, B_{s,i}, x_{s,i}) + \mathrm{E}\Big(N_{t',i} - x_{t',i}, \vec{Q}^{(1)}_{t',i}(x_{t',i}), B^{(1)}_{t',i}(x_{t',i});\ t'\Big),$$

  end for
  Train neural network $\mathrm{E}(\cdot;\ t)$ over inputs $(\vec{z}_i)$ with responses ϱ_i.
end for

7 Calibration of the Model

In this section, we calibrate the model on historical data and estimate the optimal liquidation costs for 12 firms among the most liquid and actively traded stocks on the NYSE exchange through the *Arcabook*. The calibration is done with the historical limit order books from January 4th to January 31st, 2021. Out-of-sample tests are performed with data ranging from February 1st to February 26th, 2021. The data files obtained from ICE Data Services (theice.com) contain all time-stamped limit orders entered, removed, modified, filled or partially filled on the NYSE *Arcabook* platform.

The firms differ from a statistical point of view. See Table 1 for main statistics and calibrated parameters and Fig. 1 for the steady state distribution for 6 stocks. Lot sizes are chosen to best represent the data while keeping the calibration tractable. Table 1 also reports the lot sizes, the largest quantity observed, the number of levels used for the simulation of the LOB and the number of levels used in the neural network estimation of $\mathcal{E}$, for each stock. The tick size is \$0.01.

To calibrate the model, we reconstruct the LOB and keep at least 10 levels (and up to 200 levels) under the best bid price and their associated quantities at the end of each one-second intervals during trading hours.

Table 1 Calibrated parameters. m is the number of levels simulated in the LOB. N_0^A is the number of shares in Experiment A. The last column gives the average number of levels needed to sell the number of shares N_0^A in one trade

Stock	Lot size	Max nb of lots	m	Nb of levels used for $\mathcal{E}$	Av. price	N_0^A	Nb of levels needed
AAPL	25	60	49	15	133.23	111	12
BDSI	10	430	10	5	4.39	54[a]	1[a]
CDEV	100	180	10	5	2.20	44[a]	1[a]
HSBC	100	80	16	5	27.43	19	1
JPM	10	50	33	10	134.15	92	13
MSFT	25	20	41	20	222.37	92	23
NFLX	10	40	150	20	528.87	20	56
NVDA	10	30	50	20	532.84	25	47
QQQ	100	20	32	10	318.21	161	60
GOOG	10	20	200	20	1808.13	25	173
TSLA	10	20	100	20	830.47	42	74
VXUS	1000	40	10	10	62.09	51	5

[a]For BDSI and CDEV, this number of shares is the average number on the first level, which costs more than 5bps

7.1 Benchmark Performance

The performance of the model is compared to benchmark models commonly used in the literature: TWAP, Obizhaeva and Wang (2013), Almgren and Chriss (1999), Almgren (2012), and Alfonsi et al. (2010).

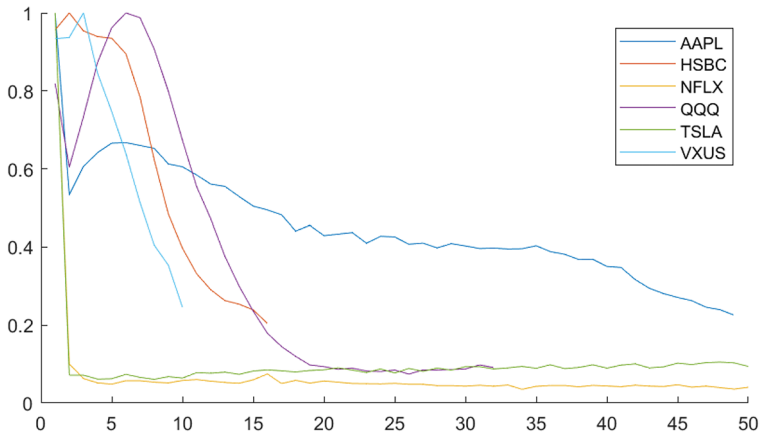

Fig. 1 Average number of shares per level for AAPL, HSBC, NFLX, QQQ, TSLA and VXUS. Maximum values are normalized to 1, for comparison purposes

7.2 Time-weighted Average Pice (TWAP)

In this strategy, the total number of shares is divided by the number of time steps:

$$x_t = \frac{N_0}{T+1}.$$

7.3 The Obizhaeva and Wang model (OW)

The OW model assumes an LOB with constant quantities at all levels and exponential resilience of price impacts. The only source of randomness is the midprice. When the number of time steps is large, the optimal strategy consists in two larger trades at time 0 and T and a series of smaller equal trades in between. We discretize the solution of the continuous-time limit, given in Proposition 2 of Obizhaeva and Wang (2013), as an approximation of the optimal trading trajectory:

$$\begin{aligned} x_0 &= \frac{N_0}{\rho T + 2}, \\ x_t &= \frac{\rho N_0}{\rho T + 2} \frac{T}{T-1}, \quad t = 1, \dots, T-1, \\ x_T &= \frac{N_0}{\rho T + 2} \end{aligned}$$

The parameter ρ cannot be directly observe and there is no straightforward approach to estimating it. We therefore use a robust benchmark performance approach in which we keep the value of ρ which minimizes the expected liquidation cost in simulations trials.

7.4 The Almgren and Chriss model (AC)

The AC model takes into account a permanent and temporary price impact of trading. The investor is assumed to optimize a mean-variance type of objective. The only source of risk is the midprice and the optimal solution is deterministic. The strategy typically trades faster at first and slows down toward the end. The optimal trading trajectory depends on a parameter κ which is a function of price impact parameters, risk aversion and market volatility. It is given by

$$x_t = N_0 \frac{\sinh(\kappa(T-t)) - \sinh(\kappa(T-t-1))}{\sinh(\kappa T)}.$$

Following Almgren and Chriss (1999), we choose $\kappa = 3/T$.

7.5 The Almgren Model of Stochastic Liquidity (A)

In Almgren (2012), the coefficient of temporary market impact η and the volatility of prices σ are assumed to be stochastic and negatively correlated:

$$\eta_t = \bar{\eta} e^{\xi_t}, \quad \sigma_t = \bar{\sigma} e^{-\xi_t/2}.$$

When ξ evolves according to an Orstein-Uhlenbeck stochastic process of the form,

$$d\xi_t = -\frac{\xi}{\delta}dt + \frac{\beta}{\sqrt{\delta}}dW_t,$$

the optimal trading speed is

$$v = \frac{N_0}{\delta}e^{-\xi}u((T-t)/\delta, \xi)$$

in which u is the solution of the following PDE:

$$u_\tau + \xi u_\xi = e^{-\xi}(K^2 - u^2) + \frac{1}{2}\beta^2 u_{\xi\xi}$$

with initial condition

$$u(\tau, \xi) = \frac{e^\xi}{\tau} - \frac{1}{2}(\xi - \frac{1}{2}\beta^2)e^\xi + O(\tau) \quad \text{as } \tau \to 0.$$

While simulating the order book, we estimate η_t as the inverse of the average order book density at time t. Given that the unconditional expectation of ξ_t is zero, we set

$$\bar{\eta} = \exp(\vec{E}\log(\eta_t))$$

and estimate this quantity by simulation. Given that the unconditional variance of η is $\frac{1}{2}\beta^2$, we set

$$\beta = \sqrt{2\ \mathrm{Var}\ (\xi_t)}$$

K is a nondimensional risk-aversion parameter. We set it equal to 0.1 in the simulations.

7.6 The Alfonsi-Fruth-Schied Model (AFS)

Alfonsi et al. (2010) present the liquidation problem in an LOB with a general density function f, fixed in time. The limit orders are assumed to recover after a trade at an exponential rate ρ. We define $F(x) = \int_0^x f(y)dy$. Theorem 4.1 states that the optimal trading trajectory is

$$\begin{aligned} x_t =& x_0(1 - e^{-\rho}), \quad t = 1, \dots, T-1 \\ x_T =& N_0 - x_0 - (T-1)x_0(1 - e^{-\rho}) \end{aligned}$$

in which x_0 is the unique solution of

$$F^{-1}(N_0 - Tx_0(1 - e^{-\rho}))(1 - e^{-\rho}) = F^{-1}(x_0) - e^{-\rho}F^{-1}(e^{-\rho}x_0).$$

The density function is estimated from the average quantities in the steady-state distribution of the LOB.

Table 2 Out-of-sample cost statistics for 10,000 simulations for each algorithm and benchmark methods for Experiment A

	NNDP1 (bps)	Std	NNDP2 (bps)	Std	TWAP	OW	AC	A	AFS
AAPL	62.2 (1.68)	57.5	41.3* (1.12)	131.9	76.9	74.8	82.1	72.1**	88.3
BDSI	6.7 (28.84)	4.6	6.6* (28.48)	4.5	8.2	8.3	8.3	7.6**	8.3
CDEV	69.2(71.28)	39.5	68.3*(70.38)	35.9	82.2	83.1	83.8	77.4**	84.4
GOOG	16.2 (0.36)	329.7	9.2* (0.20)	329.9	404.2	258.2**	388.7	258.5	325.6
HSBC	25.7 (4.94)	10.0	25.1* (4.81)	14.8	28.7	28.2	29.2	27.7**	31.3
JPM	29.1(2.36)	47.8	19.5*(1.58)	33.9	55.1	55.3	55.5	36.0**	58.4
MSFT	170.3 (3.33)	142.8	148.9*(2.91)	165.1	458.6	467.2	471.4	403.6**	455.6
NFLX	30.0* (2.83)	62.2	81.1 (7.67)	126.4	136.3	97.4	115.4	30.9**	207.4
NVDA	39.2* (2.95)	75.5	54.4 (4.09)	117.2	108.0	79.5	92.5	48.3**	215.7
QQQ	1886.0(3.68)	799.9	452.4*(0.88)	609.7	1449.1	1416.8	1390.8	1368.5**	1512.0
TSLA	73.2* (2.10)	108.1	122.1 (3.50)	215.8	320.7	246.3	290.9	178.9**	260.4
VXUS	1449.9(4.58)	507.1	1241.3*(3.92)	789.4	1705.9	1761.2	1738.5	1507.4**	1729.7

*Best method

**Best benchmark method

8 Numerical Results

In this numerical implementation, we liquidate a given number of shares over 60 and 300 seconds, at one-second intervals. For the first 60-second implementation (Experiment A), the number of shares considered corresponds to the number of shares that would incur an average cost of 5 basis points if they were liquidated in a single trade. This average single-trade cost is computed over a random sampling taken from the steady-state distribution. In the case of BDSI and CDEV, however, the number of shares liquidated simply corresponds to the average number of shares in the first level since for these two stocks 5 bps would be less than one level. In Experiment B, we liquidate 3 times this number of shares over 60 seconds. Experiment C is over a 300-second period and the number of shares liquidated corresponds to 5 times the number of shares in Experiment A.

Liquidation costs are computed by simulating the LOB, using the out-of-sample data of February 2021 to estimate the transition probabilities as described in Section 4. Out-of-sample results are presented in Tables 2, 3 and 4. Percentage improvement of NNDP over benchmark methods is reported in Table 5. In-sample costs can be found in Tables 9, 10 and 11. Overall, NNDP2 outperforms NNDP1 for all stocks in the out-of-sample tests for Experiment B, whereas NNDP1 gives better results in only 4 cases in total in the other two experiments. This result is expected since the hybrid NNDP2 method aims to diminish the accumulation of errors, resulting in a better representation of the optimal liquidation policy. On the other hand, NNDP2 is more simulation intensive and may need a higher number of simulations in order to outperform NNDP1 more consistently. The fact that NNDP2 does not consistently outperform NNDP1 in the out-of-sample tests may be evidence of over-fitting, given the good performance in the training phase. The only case where NNDP1 gives a better result than NNDP2 in the training (i.e., in-sample) phase is GOOG for Experiment C, which suggests that the number of simulations may need to be increased to have a better fit. Nevertheless, NNDP2 outperforms NNDP1 for this specific case in the out-of-sample test.

Table 3 Out-of-sample cost statistics for 10,000 simulations for each algorithm and benchmark methods for Experiment B

	NNDP1 (bps)	Std	NNDP2 (bps)	Std	TWAP	OW	AC	A	AFS
AAPL	598.7 (5.40)	303.4	273.6* (2.47)	363.7	949.1	893.1	1036.9	774.4**	967.8
BDSI	48.7 (70.05)	41.9	48.6* (69.86)	43.9	57.5	54.3	57.7	53.5**	60.6
CDEV	463.7 (159.32)	233.9	456.7* (156.91)	235.5	584.4	578.5	592.1	537.8**	589.4
GOOG	1491.4 (11.00)	1217.8	803.4* (5.92)	1368.1	1792.9	1373.5**	1635.7	1399.1	1612.0
HSBC	133.1 (8.52)	54.0	125.5* (8.03)	80.4	171.6	169.5	175.4	151.7**	172.0
JPM	138.5 (3.74)	169.4	132.1* (3.57)	176.4	563.3	563.2	588.9	331.1**	575.6
MSFT	2679.4 (17.46)	1613.1	2144.6*(13.98)	1230.9	4608.2	4470.4**	4853.8	5707.0	4818.3
NFLX	345.1 (10.87)	385.7	274.5* (8.65)	337.7	569.2	535.6	550.9	492.5**	545.0
NVDA	324.7 (8.13)	407.5	298.6* (7.47)	370.9	464.6	452.8	445.0	423.0**	469.4
QQQ	10681.7(6.95)	2092.2	4261.3*(2.77)	2683.6	9954.5**	10071.6	11327.3	26763.3	10087.9
TSLA	1128.1 (10.78)	821.8	496.0* (4.74)	621.4	1830.6	1522.6	1668.6	1247.2**	1724.5
VXUS	14562.5(15.33)	3411.4	12299.3*(12.95)	3484.8	17350.5	16276.2	18143.6	15011.2**	16491.8

*Best method

**Best benchmark method

In all cases, the best NNDP algorithm beats all the benchmark models. Even in the case of BDSI, CDEV and HSBC there is a reduction in cost despite the fact that the number of shares sold is inside the first level (best bid) on average. Although the benchmark liquidation solutions are quantitatively different, they often give similar results. The main difference between the benchmark methods (excluding the Almgren model) and the NNDP algorithms is that the latter take into account the random evolution of the order book. The main improvement from other liquidation strategies is that the NNDP algorithms consider the fast changing state of the order book and the impact that deeper levels of the order book have on the overall liquidity. Among the benchmarks, we see that Almgren (2012) dominates for shorter liquidation periods, whereas Obizhaeva and Wang (2013) can sometime give better results in Experiment B and C. The former takes into account the stochastic evolution of liquidity whereas the latter takes into account the (average) shape of the LOB. The NNDP approach shows that taking into account both the shape and its random evolution through time can significantly improve the performance of the optimal solution.

Table 4 Out-of-sample cost statistics for 10,000 simulations for each algorithm and benchmark methods for Experiment C

	NNDP1 (bps)	Std	NNDP2 (bps)	Std	TWAP	OW	AC	A	AFS
AAPL	762.8(4.13)	980.9	645.4* (3.49)	1439.9	1414.7**	1445.6	2989.1	4527.9	1595.9
BDSI	84.2 (72.62)	72.2	69.9* (60.24)	97.6	136.9	134.8	167.3	87.9**	132.5
CDEV	938.2(193.43)	474.5	911.1* (187.83)	528.6	1389.2	1391.4	1817.1	1028.8**	1485.5
GOOG	392.2(1.74)	4371.8	339.2* (1.50)	4657.0	5539.7	2501.7**	3149.2	2802.4	3074.3
HSBC	300.9(11.55)	137.5	245.8* (9.44)	304.1	379.2	385.1	438.9	352.6**	469.0
JPM	301.3*(4.88)	429.6	437.8 (7.10)	648.8	1300.0**	1302.8	2200.0	1330.6	1510.8
MSFT	4203.1(16.44)	2153.2	4118.5*(16.10)	2889.3	10431.2	10304.5**	14246.0	24841.3	11394.6
NFLX	2738.9(51.79)	1402.6	1123.8*(21.25)	921.4	2101.6	1515.5	1260.0	1224.0**	5313.7
NVDA	1083.2(16.26)	1097.7	1064.9* (15.99)	1071.3	2028.5	1505.4	1113.9	1100.2**	5741.2
QQQ	18279.6(7.14)	8189.9	6915.3* (2.70)	8584.9	31631.4	31601.4	34747.2	38425.0	30979.3**
TSLA	4323.2(24.79)	1811.5	2024.0*(11.60)	2505.3	7481.9	5506.7	4029.9	2950.2**	6196.8
VXUS	29267.5(18.48)	7950.0	27560.2*(17.41)	8466.6	36381.8	36197.2**	60611.1	48596.3	37838.2

Table 5 Percentage improvement of NNDP over benchmark methods

	Experiment A		Experiment B		Experiment C	
	2nd best diff[a]	Ave. diff[b]	2nd best diff[a]	Ave. diff[b]	2nd best diff[a]	Ave. diff[b]
AAPL	43%	48%	59%	66%	54%	73%
BDSI	13%	19%	9%	14%	21%	47%
CDEV	12%	17%	15%	21%	11%	36%
GOOG	96%	97%	42%	49%	86%	91%
HSBC	10%	14%	17%	25%	30%	39%
JPM	46%	62%	60%	75%	77%	80%
MSFT	63%	67%	52%	56%	60%	71%
NFLX	3%	76%	44%	49%	8%	51%
NVDA	19%	66%	29%	34%	3%	54%
QQQ	67%	68%	57%	69%	78%	91%
TSLA	59%	72%	60%	69%	31%	61%
VXUS	18%	26%	18%	26%	24%	37%

[a]difference between NNDP cost and best benchmark method

[b]difference between NNDP cost and average cost of all benchmark methods

Figure 2 depicts the percentiles of the remaining number of shares to liquidate, calculated at each point in time for six selected stocks in Experiment B. We see that the optimal strategy varies from one stock to the other. For NFLX and HSBC, optimal trading strategies have a lot more variation. Between 10 and 25% of trajectories for AAPL and HSBC terminate before the end of the period. This suggests that liquidity is high in some simulations and a higher number of shares can be liquidated with limited price impact. On the other hand, the optimal liquidation strategies of QQQ are much more constant with little variation. The fact that QQQ acheives a 2.77 bps cost suggests that liquidity is high and constant through time. Yet, the NNDP2 method decreases costs by 57% the cost associated with the TWAP method, the best benchmark methods for this specific case. For most stocks, many trading trajectories exhibit a large trade at the end. This is typical of a market with high resilience as modelled by the AFS model with an optimal solution which often has a large jump at the end. Yet, AFS only outperformed the other benchmarks once in all three experiments.

The distribution of liquidation costs of the same six stocks is presented in Fig. 3 along with the best benchmark method in each case. For NFLX, almost half of the simulations give a negative liquidation cost. Once again, this shows that posted liquidity (i.e, limit orders present in the LOB) is high with respect to the observed average volume so that the price impact of an average trade is low. In all cases, we see that the average cost is lower and the largest costs for NNDP2 (i.e., the right tail of the distribution) is lower or equal to the tail of the best benchmark model, except for NFLX which has a few more extreme cases. Table 6 presents the value at risk (VaR) of liquidation costs at the 95% level. We see that VaR is higher for NNDP in only three cases (all in Experiment B), even though risk is not taking into account in the minimization procedure. Furthermore, we see in Table 7 that in almost all cases the risk-adjusted liquidation costs, define as the ratio between the average costs and their standard deviation, is lower for NNDP than the best benchmark model. Overall, the risk-adjusted costs are 38%, 58% and 57% lower for NNDP than the best benchmark for Experiments A, B and C, respectively.

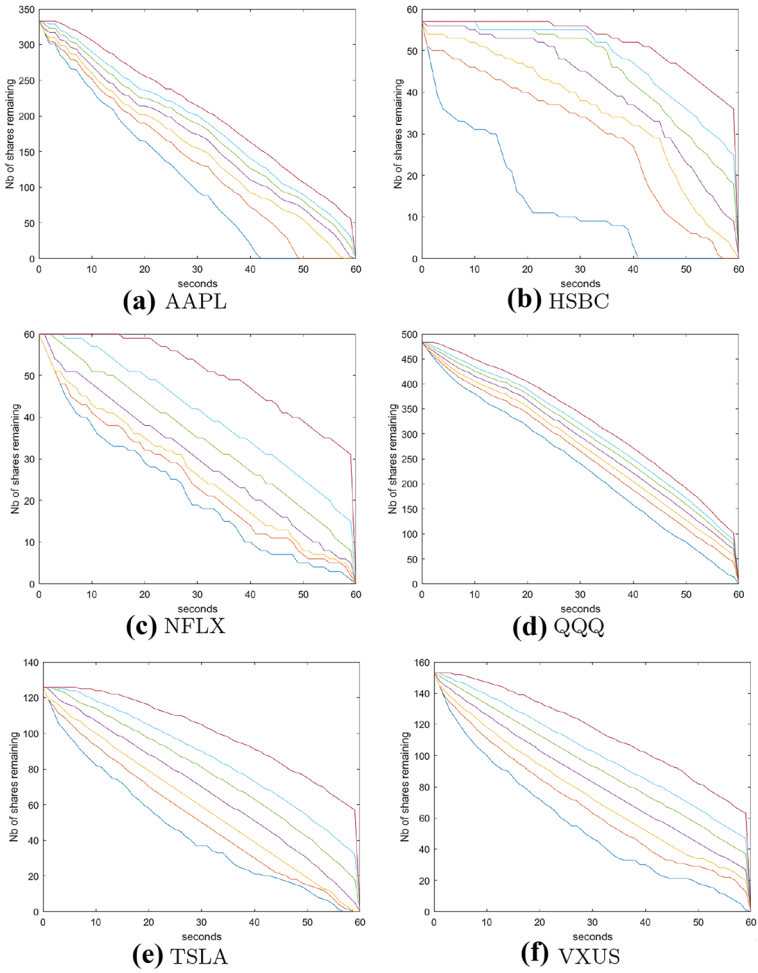

Fig. 2 Percentiles of the number of shares remaining (in lots) of the optimal liquidation solution. From top to bottom, the curves represent the 1st, 10th, 25th, 50th, 75th, 90th and 99th percentiles

To have a better understanding of the importance of the deeper levels of the LOB on the liquidation strategy, Table 8 shows the average liquidation cost for AAPL in Experiment B, associated to strategies that can only depend on the first x levels. The simulation of the evolution of the LOB is done with all the levels, but the expectation of the future value function is done with respect to the current number of shares remaining, and the first x levels. Table 8 shows a clear decrease in the liquidation cost when the number of levels considered increases, with in-sample cost leveling at 15 levels for NNDP2 and a constant decrease for NNDP1. Out-of-sample costs increase with 49 levels for NNDP2, which may be evidence of overfitting.

Table 12 presents the computing time of the algorithms. Results were obtained using the Cedar Compute Canada cloud computing resource with a 48-core CPU with 2.1 GHz speed. The calibration of the neural networks and the minimization over the set of actions can be done relatively efficiently. We see that the most computationally

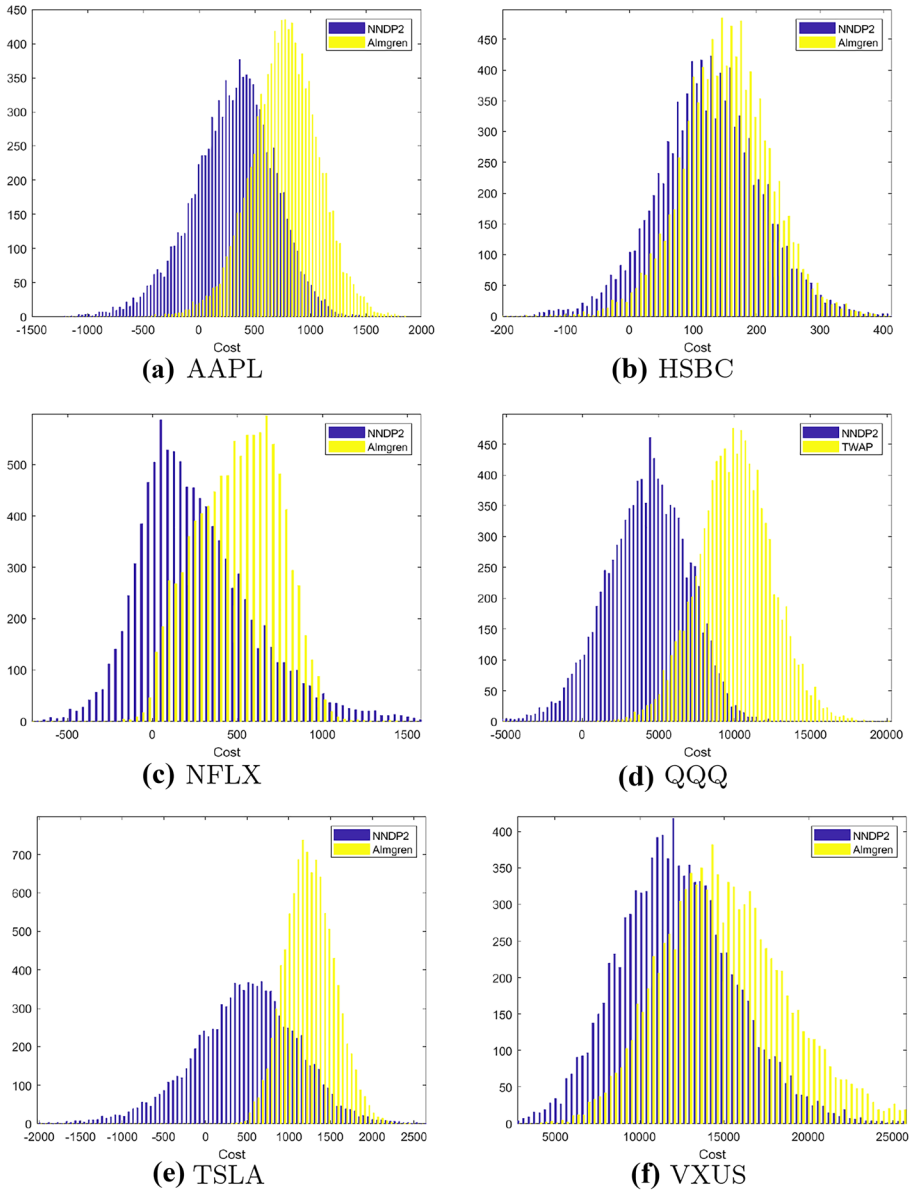

Fig. 3 Distribution of liquidation costs compared to the best benchmark method

expensive parts of the algorithms are the simulation (which includes the random sampling of the state space and its random evolution) and the computation of the impact function (i.e. the impact of a trade on the state of the LOB, including the relabeling of the levels). In total, the computing time of the algorithm is approximately 4 times larger for Experiment C than A, which is expected since the former has 5 times more timesteps. However, computing time for NNDP2 is only 4 times more than NNDP1

Table 6 Comparison of VaR for NNDP and best benchmark model for all three experiments

	Experiment A		Experiment B		Experiment C	
	NNDP	Benchmark	NNDP	Benchmark	NNDP	Benchmark
AAPL	247,0*	263,3	872,0*	1247,5	2948,3*	3681,5
BDSI	14,2*	20,6	133,7*	138,4	242,6*	619,4
CDEV	132,0*	160,0	895,0*	1000,0	1825,0*	2884,0
GOOG	629,4*	1333,4	3188,4	1679,9*	8073,6*	14174,6
HSBC	49,0*	80,0	259,0*	263,0	754,0*	1135,0
JPM	75,1*	145,7	430,3*	639,9	1008,2*	2865,9
MSFT	418,5*	825,5	4245,0*	6633,0	9173,8*	16600,5
NFLX	224,5*	545,5	907,1	871,2*	2813,1*	8728,7
NVDA	181,3*	526,3	933,4	630,3*	2925,1*	8840,2
QQQ	1511,0*	2821,0	8441,0*	13920,0	20726,0*	43409,0
TSLA	239,1*	505,5	1447,9*	1749,3	5016,3*	8548,1
VXUS	2850,0*	2900,0	18250,0*	21970,0	41970,0*	54530,0

*Lowest VaR

even though each simulated path has ten times more timesteps ($\omega = 10$). We also see that stocks like GOOG and NFLX have larger computing times for the simulation part since they have considerably more levels in the order books.

Table 7 Comparison of risk-adjusted costs for NNDP and best benchmark model for all three experiments

	Experiment A		Experiment B		Experiment C	
	NNDP	Benchmark	NNDP	Benchmark	NNDP	Benchmark
AAPL	0,31*	0,83	0,87*	2,54	0,45*	1,35
BDSI	1,45	1,01*	1,11*	1,29	0,72*	0,77
CDEV	1,90*	1,99	1,98*	2,17	1,72*	2,14
GOOG	0,03*	0,54	0,59*	6,94	0,07*	0,41
HSBC	1,70	1,06*	1,56*	2,19	0,81*	1,21
JPM	0,58*	1,11	0,75*	1,88	0,70*	1,91
MSFT	0,90*	1,90	1,74*	3,82	1,43*	3,79
NFLX	0,54*	1,44	0,81*	2,06	1,22*	3,04
NVDA	0,52*	1,48	0,80*	3,23	0,99*	3,09
QQQ	0,84*	1,82	1,59*	4,18	0,81*	4,14
TSLA	0,68*	2,00	0,80*	4,24	1,07*	4,63
VXUS	1,51*	2,58	3,53*	3,91	3,26*	4,02

*Lowest VaR

Table 8 Liquidation costs of AAPL when the trading strategy only depends on the first x levels

# of levels considered	NNDP1		NNDP2	
	In-sample	Out-of-sample	In-sample	Out-of-sample
0*	1681.13	1268.67	1452.89	1151.26
0**	679.23	607.02	494.17	459.31
1	639.18	591.85	445.89	382.85
2	650.64	616.23	434.28	363.89
3	638.34	606.11	405.46	377.78
5	644.36	612.27	340.51	315.20
10	632.21	589.58	366.04	286.38
15	609.69	565.24	356.28	273.55
49	552.19	438.52	363.87	315.89

*Only the remaining number of shares taken into account

**Only the remaining number of shares and the current best bid price taken into account

Table 9 In-sample cost statistics for 10,000 simulations for each algorithm and benchmark methods for Experiment A

	NNDP1 (bps)	Std	NNDP2 (bps)	Std	TWAP	OW	AC	A	AFS
AAPL	64.6 (1.75)	67.0	47.7* (1.29)	151.9	104.4	104.4	108.1	70.6**	112.5
BDSI	6.6 (28.21)	3.6	6.3* (27.29)	3.4	7.5	7.5	7.7	7.1**	7.7
CDEV	48.9 (50.38)	22.2	48.4* (49.85)	22.9	59.6	58.9	59.8	58.1**	59.5
GOOG	11.2 (0.25)	150.8	7.7* (0.17)	155.3	161.1	149.0	170.8	93.4**	160.0
HSBC	22.5 (4.31)	8.7	22.4* (4.29)	16.2	25.5	24.2	25.6	25.5	25.3**
JPM	15.2 (1.23)	49.3	11.5* (0.93)	34.2	40.5	41.4	42.3	20.5**	42.3
MSFT	89.9 (1.76)	91.8	48.9* (0.96)	102.6	150.8	155.5	175.2	85.5**	465.5
NFLX	13.3 (1.26)	63.9	3.2* (0.30)	85.5	60.9	46.4	58.1	7.1**	59.5
NVDA	18.6 (1.40)	71.8	9.1* (0.68)	115.0	54.4	51.6	54.0	28.4**	55.1
QQQ	2063.9(4.03)	809.2	446.0*(0.87)	626.0	1686.1	1622.5	1581.7**	2784.5	1744.9
TSLA	45.9 (1.32)	130.5	1.2* (0.03)	231.2	270.8	163.5**	253.2	169.1	232.5
VXUS	1245.8(3.93)	500.1	1143.0*(3.61)	944.9	1454.0	1427.5	1479.4	1285.9**	1471.1

*Best method

Best benchmark method

Table 10 In-sample cost statistics for 10,000 simulations for each algorithm and benchmark methods for Experiment B

	NNDP1 (bps)	Std Dev.	NNDP2 (bps)	Std Dev.	TWAP	OW	AC	A	AFS
AAPL	621.6 (5.60)	357.6	356.9* (3.22)	408.7	1243.2	1179.3	1333.0	772.5**	1281.0
BDSI	44.6 (64.09)	35.4	43.1* (61.98)	36.0	51.7	50.7	52.4	50.7**	57.1
CDEV	220.4 (75.73)	95.7	219.0* (75.25)	96.0	299.5	292.8	302.1	290.8**	305.2
GOOG	636.3 (4.69)	901.0	363.8* (2.68)	809.0	1235.6	1219.4	1230.7	1206.9**	1213.8
HSBC	106.3 (6.80)	47.2	93.7* (6.00)	73.2	129.6	130.1	132.6	116.5**	130.6
JPM	26.4 (0.71)	157.5	26.3* (0.74)	155.3	381.6	387.6	428.8	199.7**	377.9
MSFT	1536.1 (10.01)	458.1	437.4* (2.85)	514.2	2243.2	2024.2	2267.0	1472.8**	1880.7
NFLX	178.3 (5.62)	303.6	30.8* (0.97)	312.6	357.4	346.1**	372.0	358.9	360.4
NVDA	162.2 (4.06)	397.6	106.1* (2.65)	360.8	388.6	384.5	387.4	389.2	382.6**
QQQ	11730.4(7.63)	2252.1	4543.1*(2.96)	2797.2	11032.5**	11256.3	12596.2	73827.2	11233.2
TSLA	863.0 (8.25)	868.8	232.2* (2.22)	705.5	1539.4	1171.6**	1447.0	1193.7	1490.5
VXUS	12312.2(12.96)	3736.8	10221.1*(10.76)	3568.2	14674.4	13674.5	15016.8	12881.9**	13875.9

*Best method

Best benchmark method

Table 11 In-sample cost statistics for 10,000 simulations for each algorithm and benchmark methods for Experiment C

	NNDP1 (bps)	Std Dev.	NNDP2 (bps)	Std Dev.	TWAP	OW	AC	A	AFS
AAPL	843.7 (4.56)	1128.8	676.2* (3.66)	1678.3	2144.8	2123.9**	4007.3	2306.9	2251.6
BDSI	69.2 (59.65)	49.3	57.1* (49.25)	63.8	110.5	109.3	147.1	85.0**	131.6
CDEV	425.3 (87.68)	260.6	421.0* (88.86)	277.8	661.2	670.9	753.6	564.1**	755.7
GOOG	1831.7* (8.10)	2709.9	2058.6 (9.11)	2012.0	3791.6	3115.9	2788.2	2638.3**	4749.7
HSBC	241.4 (9.27)	128.4	160.8* (6.17)	247.3	279.3	278.1	325.9	222.0**	344.1
JPM	213.1 (3.45)	417.2	81.2* (1.32)	480.6	843.4	886.4	1551.3	686.1**	915.6
MSFT	1879.6 (7.35)	1350.8	471.8* (1.84)	1884.3	3482.0	3398.2**	7638.5	6107.0	3764.7
NFLX	356.0 (6.73)	1432.0	243.2* (4.60)	883.5	1072.7	1007.0**	1006.4	1046.9	1356.2
NVDA	63.2 (0.95)	1032.1	56.2* (0.84)	1048.5	952.8	916.0	959.72	1077.2	834.1**
QQQ	20731.0 (8.09)	8613.5	7031.6*(2.75)	9003.0	38864.1	38654.9	38342.0	248585.5	38010.6**
TSLA	3766.0 (21.59)	1387.4	1123.0*(6.44)	2081.3	6727.7	3862.2	3714.3	2896.0**	5638.5
VXUS	26556.7 (16.77)	9125.7	24461.0*(15.45)	9451.0	31718.7	32298.9	53280.5	24483.6**	33054.3

*Best method

Best benchmark method

Table 12 Computing times in seconds

	Experiment A - NNDP1					Experiment C - NNDP1					Experiment A - NNDP2				
	NN[a]	MIN[b]	SIM[c]	IMP[d]	Total	NN	MIN	SIM	IMP	Total	NN	MIN	SIM	IMP	Total
AAPL	5.9	4.63	30.6	11.5	52.6	25.6	10.79	123.7	12.7	172.8	7.2	28.57	117.5	69.3	222.6
BDSI	26.0	2.61	44.3	1.3	74.2	17.6	7.58	198.7	2.0	225.9	28.2	15.00	158.4	6.1	207.7
CDEV	3.6	2.66	19.5	1.4	27.2	64.9	7.14	72.2	1.6	145.8	4.5	12.29	67.1	4.0	87.9
GOOG	5.6	1.67	290.0	10.0	307.3	84.8	6.70	1468.3	19.5	1579.3	7.5	17.56	2151.7	161.8	2338.6
HSBC	4.5	2.51	13.3	1.6	21.9	31.3	7.39	49.1	2.5	90.3	5.6	13.25	36.7	7.1	62.7
JPM	5.9	2.96	19.2	8.1	36.2	21.5	7.27	73.9	6.4	109.1	7.5	22.07	65.8	54.2	149.6
MSFT	5.9	3.07	18.7	9.5	37.2	26.2	7.26	71.6	7.1	112.2	6.1	23.18	50.5	60.7	140.5
NFLX	5.2	1.84	117.5	9.0	133.5	25.7	5.91	567.6	12.5	611.7	6.6	15.21	673.1	98.0	792.9
NVDA	4.8	1.52	42.6	4.3	53.2	27.7	5.53	202.7	4.7	240.6	5.7	13.79	244.3	32.9	296.7
QQQ	21.2	2.29	14.5	7.4	45.4	106.8	7.17	55.6	6.2	175.8	24.0	18.68	35.7	49.7	128.1
TSLA	6.8	2.28	86.6	30.3	126.0	27.7	6.10	412.6	19.4	465.8	7.4	19.87	516.9	230.7	774.9
VXUS	12.5	2.12	10.8	1.5	26.9	24.7	7.05	40.4	1.8	74.0	12.4	13.22	23.5	7.7	56.8

[a]Calibration of neural networks

[b]Minimization operation over actions

[c]Simulation part of algorithm

[d]Price impact and relabeling operation

9 Conclusion

This paper presents a hybrid value and performance iteration algorithm based on Monte Carlo simulations and neural networks for the optimal liquidation of a fixed number of shares of an asset through a limit order book. It uses neural network techniques to represent the current state of the order book and calculates the optimal number of shares to trade at each point in time. The model was calibrated on historical LOB data and consistently and significantly outperforms common liquidation strategies in the literature. The neural network technique presented in this paper can be used for a variety of complex financial control problems with potentially large state spaces such as optimal market making, portfolio optimization and other risk management problems, with discrete and low-dimensional control space.

Funding The author's research is partly supported by the Natural Sciences and Engineering Research Council of Canada (NSERC), under grant number RGPIN-2021-03299.

Data Availability The data that support the findings of this study are available from ICE Data Services but restrictions apply to the availability of these data, which were used under licence for the current study, and so are not publicly available. Data are however available from the authors upon reasonable request and with permission of ICE Data Services.

Declarations

Conflicts of Interest The author certifies that he has no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

References

Ackermann J, Kruse T, Urusov M (2021) Càdlàg semimartingale strategies for optimal trade execution in stochastic order book models. Finance and Stochastics 25:757–810

Akbarzadeh N, Tekin C, Van der Schaar M (2018) Online Learning in Limit Order Book Trade Execution. IEEE Trans Signal Process 66:4626–4641

Alfonsi A, Fruth A, Schied A (2010) Optimal execution strategies in limit order books with general shape functions. Quantitative Finance 10:143–157

Almgren R (2012) Optimal trading with stochastic liquidity and volatility. SIAM J Financ Math 3:163–181

Almgren R, Chriss N (1999) Value under liquidation. Risk December 1999

Bachouch A, Huré C, Pham H, Langrené N (2022) Deep neural networks algorithms for stochastic control problems on finite horizon: numerical computations. Methodol Comput Appl Prob 24:143–178

Barrera-Esteve C, Bergeret F, Dossal C, Gobet E, Meziou A, Munos R, Reboul-Salze D (2006) Numerical methods for the pricing of swing options: a stochastic control approach. Methodol Comput Appl Prob 8:517–540

Bayraktar E, Ludkovski M (2014) Liquidation in limit order books with controlled intensity. Math Financ 24:627–650

Becherer D, Bilarev T, Frentrup P (2018) Optimal liquidation under stochastic liquidity. Finance and Stochastics 22:39–68

Belomestny D, Hafner S, Urusov M (2018) Regression-based complexity reduction of the nested Monte Carlo methods. SIAM Journal on Financial Mathematics 9:665–689

Bertsekas D, Tsitsiklis J (1995) Neuro-dynamic programming: an overview. Proceedings of 1995 34th IEEE Conference on Decision and Control: 560–564

Broadie M, Glasserman P, Ha Z (2000) Pricing american options by simulation using a stochastic mesh with optimized weights. In Probabilistic constrained optimization (pp. 26–44). Springer, Boston, MA

Buehler H, Gonon L, Teichmann J, Wood B (2019) Deep hedging. Quant Financ 19:1271–1291

Carbonneau A, Godin F (2021) Equal risk pricing of derivatives with deep hedging. Quant Financ 21:593–608
Carmona R, Ludkovski M (2008) Pricing asset scheduling flexibility using optimal switching. Appl Math Financ 15:405–447
Cartea A, Jaimungal S (2015) Optimal execution with limit and market orders. Quant Financ 15:1279–1291
Cartea A, Jaimungal S, Penalva J (2015) Algorithmic and high-frequency trading. Cambridge University Press
Cartea A, Jaimungal S, Ricci J (2014) Buy low, sell high: A high frequency trading perspective. SIAM Journal on Financial Mathematics 5:415–444
Cartea A, Jaimungal S, Sanchez-Betancourt L (2021) Deep reinforcement learning for algorithmic trading. https://ssrn.com/abstract=3812473
Chevalier E, Ly Vath V, Roch A, Scotti S (2016) Optimal execution cost for liquidation through a limit order market. Int J Theo Appl Financ 19(1). https://doi.org/10.1142/S0219024916500047
Cont R, Stoikov S, Talrej R (2010) A Stochastic Model for Order Book Dynamics. Oper Res 58(3):549–563
Denault M, Delage E, Simonato J-G (2017) Dynamic portfolio choice: a simulation-and-regression approach. Optim Eng 18:369–406
Denault M, Simonato J-G (2017) Dynamic portfolio choices by simulation-and-regression: Revisiting the issue of value function vs portfolio weight recursions. Comp Oper Res 79:174–189
Denault M, Simonato J-G, Stentoft L (2013) A simulation-and-regression approach for stochastic dynamic programs with endogenous state variables. Comp Oper Res 40:2760–2769
Egloff D, Kohler M, Todorovic N (2007) A dynamic look-ahead Monte Carlo algorithm for pricing Bermudan options. Ann Appl Probab 17:1138–1171
Germain M, Pham H, Warin X (2021) Neural networks-based algorithms for stochastic control and PDEs in finance. https://doi.org/10.48550/arXiv.2101.08068
Glasserman P, Yu B (2004) Simulation for american options: regression now or regression later? in: niederreiter h (ed) monte carlo and quasi-monte carlo methods 2002. Springer, Berlin, Heidelberg
Guéant O, Lehalle C (2015) General intensity shapes in optimal liquidation. Math Financ 25:457–495
Guéant O, Lehalle C, Tapia J (2011) Optimal portfolio liquidation with limit orders. SIAM J Finan Math 3:740–764
Guilbaud F, Pham H (2013) Optimal high frequency trading with limit and market orders. Quant Financ 13:79–94
Gyurko LG, Hambly BM, Witte JH (2015) Monte Carlo methods via a dual approach for some discrete time stochastic control problems. Mathematical Methods of Operations Research 81:109–135
Han J, Ee W (2016) 2016. Deep learning approximation for stochastic control problems. deep reinforcement learning Workshop, NIPS
Huang YT, Kwok YK (2016) Regression-based monte carlo methods for stochastic control models: variable annuities with lifelong guarantees. Quant Finance 16:905–928
Huré C, Pham H, Bachouch A, Langrené N (2021) Deep neural networks algorithms for stochastic control problems on finite horizon: convergence analysis. SIAM J Numer Anal 59:525–557
Kharroubi I, Langrené N, Pham H (2014) A numerical algorithm for fully nonlinear HJB equations: an approach by control randomization. Monte Carlo Methods Appl 20:145–165
Lokka A, Xu J (2020) Optimal liquidation trajectories for the Almgren-Chriss Model. Int J Theo Appl Fin 23. https://doi.org/10.1142/S0219024920500491
Longstaff F, Schwartz E (2001) Valuing american options by simulation: a simple least-squares approach. Rev Financ Stud 14:113–147
Marzban S, Delage E, Li JYM (2021) Equal risk pricing and hedging of financial derivatives with convex risk measures. Quantitative Finance 22:1–27
Nevmyvaka Y, Feng Y, Kearns M (2006) Reinforcement learning for optimized trade execution. Proceedings of the 23rd international conference on Machine learning 673–680
Obizhaeva A, Wang J (2013) Optimal trading strategy and supply/demand dynamics. J Financ Mark 16:1–32
Sadoghi A, Vecer J (2022) Optimal liquidation problem in illiquid markets. European Journal of Operational Research 296:1050–1066
Schied A, Schöneborn T (2009) Risk aversion and the dynamics of optimal liquidation strategies in illiquid markets. Finance and Stochastics 13:181–204
Sirignano J (2019) Deep learning for limit order books. Quantitative Finance 19:549–570
Stentoft L (2004) Convergence of the Least Squares Monte Carlo Approach to American Option Valuation. Manag Sci 50(9):1193–1203
Sutton RS (1988) Learning to predict by the methods of temporal differences. Machine Learning 3:9–44
Tamar A, Chow Y, Ghavamzadeh M, Mannor S (2015) Policy gradient for coherent risk measures. Proceedings of the 28th International Conference on Neural Information Processing Systems: 1468–1476

Tesauro G (1992) Practical issues in temporal difference learning. Machine Learning 8:257–277
Tsitsiklis JN (1994) Asynchronous stochastic approximation and Q-learning. Machine Learning 16:185–202
Tsitsiklis JN, Van Roy B (2001) Regression methods for pricing complex American-style options. IEEE Trans Neural Netw 12:694–703
Watkins C, Dayan P (1992) Q-learning. Machine Learning 8:279–292

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.